- [x] Learn
- [x] Practice
- [x] Build Projects
- [x] Grow

Python Tuples

Immutable Collections, Endless Possibilities

Simple
Powerful
Versatile

Satyarthi

1 What is a Tuple?

A tuple in Python is an ordered collection of items that are immutable (cannot be changed).
Tuples can store different data types and allow duplicate values. They are written using round brackets ( ).

2 Creating a Tuple

You can create a tuple by placing items inside round brackets ().

Code :

```
1  # Tuple with different data types
2  student = ("Satyarthi", 19, 5.8)
3  print(student)
4  print(type(student))
```

Output

```
('Satyarthi', 19, 5.8)
<class 'tuple'>
```

3 Accessing Tuple Elements

You can access tuple elements using indexing.

Code :

```
1  fruits = ("apple", "banana", "mango")
2  print(fruits[0])   # first element
3  print(fruits[1])   # second element
4  print(fruits[-1])  # last element
```

Output

```
apple
banana
mango
```

4 Tuple Operations

You can perform basic operations on tuples.

Code :

```
1  t1 = (1, 2, 3)
2  t2 = (4, 5)
3  print(t1 + t2)    # concatenation
4  print(t1 * 2)     # repetition
5  print(2 in t1)    # membership
```

Output

```
(1, 2, 3, 4, 5)
(1, 2, 3, 1, 2, 3)
True
```

Why Use Tuples?

- Immutable (data cannot be changed)
- Faster than lists
- Can store different data types
- Allows duplicate values
- Useful for fixed data (e.g., coordinates, configuration, etc.)
- Used in functions and returning multiple values

Common Tuple Methods

Method	Description
count(x)	Returns the number of times x appears
index(x)	Returns the index of the first occurrence of x
len(t)	Returns the number of items in the tuple

Did You Know?

A tuple with a single element must have a comma!

```
t = (5,)  # This is a tuple
print(type(t))  # <class 'tuple'>
```

Key Takeaways

- [ ] Tuples are ordered and immutable.
- [ ] Use round brackets ( ).
- [ ] Can store different data types.
- [ ] Allow duplicate values.
- [ ] Useful for fixed data and function returns.
- [ ] Have useful methods like count(), index(), and len().
- [ ] Simple yet powerful!

"Small and immutable, but big on usefulness." – Satyarthi

Python Tuples

Immutable Collections, Endless Possibilities

Simple
Powerful
Versatile

1 What is a Tuple?

A tuple is an ordered collection of items that are immutable (cannot be changed). Tuples can store different data types and allow duplicate values.

2 Syntax

```
tuple_name = (item1, item2, item3)
```

Example:

```
t = (10, "Python", 3.14, True)
```

3 Key Points

- Ordered and indexed
- Immutable (cannot be changed)
- Allows duplicate values
- Can store different data types
- Written using round brackets ( )
- Useful for fixed data

4 Example

```
t = (1, "apple", 3.5, True)
print(t)
print(t[0])     # first element
print(t[2])     # third element
print(len(t))   # number of items
```

Output:

```
(1, 'apple', 3.5, True)
1
3.5
4
```

5 Practice Question

Consider the following tuple:

```
t = (10, 20, 30, 20)
```

What will be the output of the following code?

```
print(t.count(20))
print(t.index(30))
```

A
1
2

B
2
3

C
2
2

D
3
1

Think and Solve!

"Tuples keep your data safe and steady."

– Satyarthi

Python Tuples

Immutable Collections, Endless Possibilities

Simple
Powerful
Versatile

1 What is a Tuple?

A tuple is an ordered collection of items that are immutable (cannot be changed). Tuples can store different data types and allow duplicate values.

2 Syntax

```
tuple_name = (item1, item2, item3)
```

Example:

```
t = (1, "Python", 3.14, True)
```

3 Key Points

- Ordered and indexed
- Immutable (cannot be changed)
- Allows duplicate values
- Can store different data types
- Written using round brackets ( )
- Useful for fixed data

4 Example

```
t = (""apple", 10, 3.5, True)
print(t)          # whole tuple
print(t[1])       # second element
print(t[-1])      # last element
print(len(t))     # number of items
```

Output:

```
('apple', 10, 3.5, True)
10
True
4
```

5 Practice Question

Consider the following tuple:

```
t = (5, 10, 15, 20, 25)
```

What will be the output of the following code?

```
print(t[1:4])
```

A (10, 15)

B (10, 15, 20)

C [10, 15, 20]

D (15, 20)

Think and Solve!

"Small tuples, big possibilities."

– Satyarthi

Python Tuples

Immutable Collections, Endless Possibilities

Simple
Powerful
Versatile

Satyarthi

1 What is a Tuple?

A tuple is an ordered collection of items that are immutable (cannot be changed). Tuples can store different data types and allow duplicate values.

2 Syntax

```
tuple_name = (item1, item2, item3)
```

Example:

```
t = (1, "Python", 3.14, True)
```

3 Key Points

- Ordered and indexed
- Immutable (cannot be changed)
- Allows duplicate values
- Can store different data types
- Written using round brackets ( )
- Useful for fixed data

4 Example

```
t = (10, 20, 30)
print(t[0])    # first element
print(t[2])    # third element
print(len(t))  # number of items
```

Output:

```
10
30
3
```

5 Practice Question

Consider the following tuple:

```
t = ("red", "green", "blue", "green")
```

What will be the output of the following code?

```
print(t.count("green"))
```

A 1

B 2

C 3

D Error

Think and Solve!

"Tuples are small, but they hold big power." – Satyarthi

Python Tuples

Immutable Collections, Endless Possibilities

Simple
Powerful
Versatile

1 What is a Tuple?

A tuple is an ordered collection of items that are immutable (cannot be changed). Tuples can store different data types and allow duplicate values.

2 Syntax

```
tuple_name = (item1, item2, item3)
```

Example:

```
t = (1, "Python", 3.14, True)
```

3 Key Points

- Ordered and indexed
- Immutable (cannot be changed)
- Allows duplicate values
- Can store different data types
- Written using round brackets ( )
- Useful for fixed data

4 Example

```
t = (""apple", 10, 3.5, True)
print(t)          # whole tuple
print(t[1])       # second element
print(t[-1])      # last element
print(len(t))     # number of items
```

Output:

```
('apple', 10, 3.5, True)
10
True
4
```

5 Practice Question

Consider the following tuple:

```
t = (5, 10, 15, 20, 25)
```

What will be the output of the following code?

```
print(t[1:4])
```

A (10, 15)

B (10, 15, 20)

C [10, 15, 20]

D (15, 20)

Think and Solve!

"Small tuples, big possibilities."

– Satyarthi

- [x] Learn
- [x] Practice
- [x] Build Projects
- [x] Grow

Python Strings

Text Today, Possibilities Tomorrow

Simple
Powerful
Versatile

Satyarthi

1 What is a String?

A string in Python is a sequence of characters used to store text. Strings are immutable (cannot be changed) and are written inside single (`''`), double (`""`) or triple quotes (`'''''''`).

2 Creating a String

Code :

```
s1 = 'Hello'
s2 = "Python"
s3 = '''Learn
      Python'''
```

3 Accessing Characters

You can access characters using indexing (starts from 0) and negative indexing.

Code :

```
s = "Python"
print(s[0])   # P
print(s[-1])  # n
```

4 Common String Methods

Method	Description
`len(s)`	Returns the length of the string
`s.lower()`	Converts to lowercase
`s.upper()`	Converts to uppercase
`s.strip()`	Removes leading and trailing spaces
`s.replace(a, b)`	Replaces a with b
`s.split()`	Splits the string into a list
`s.find(x)`	Returns the index of x (or -1)
`s.count(x)`	Counts occurrences of x

5 Example

Code :

```
s = "  Python  "
print(len(s))
print(s.strip())
print(s.upper())
print(s.replace("Python",
"Programming"))
```

Output:

```
10
Python
PYTHON
Programming
```

6 String Operations

Code :

```
a = "Hello"
b = "World"
print(a + " "+ b) # Concatenation
print(a * 3)      # Repetition
```

Output:

```
Hello World
HelloHelloHello
```

7 Practice Question

Consider the following string:

```
s = "Python Programming"
```

What will be the output of the following code?

```
print(s[7:18])
print(s.upper().count("P"))
```

A. Programming
2

B. Programming
1

C. Program
2

D. Program
1

Think and Solve!

8 Key Takeaways

- [x] Strings store text data.
- [x] Strings are immutable.
- [x] Use indexing and slicing to access characters.
- [x] Many built-in methods make string handling easy.
- [x] Strings can be concatenated and repeated.
- [x] Useful for real-world applications like data processing, web development and file handling.

"A single string can start a big journey." – Satyarthi

- [x] Learn
- [x] Practice
- [x] Build Logic
- [x] Improve

Python

Comparison Operators

Compare values and get True or False

Small Steps Big Programs

Satyarthi

1 What are Comparison Operators?

Comparison operators are used to compare two values in Python. They check the relationship between values and return either True or False.

2 Why Use Comparison Operators?

- Helps in decision making
- Used in conditional statements (if, elif, else)
- Useful in loops and program logic
- Helps to compare numbers, strings, variables, etc.
- Essential for real-world problem solving

3 List of Comparison Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater than or equal to	5 >= 5	True
<=	Less than or equal to	5 <= 2	False

4 Example Program

Code :

```
1  a = 10
2  b = 5
3  print(a == b)     # False
4  print(a != b)     # True
5  print(a > b)      # True
6  print(a < b)      # False
7  print(a >= b)     # True
8  print(a <= b)     # False
```

5 More Examples

Expression	Result
10 == 10	True
10 != 20	True
15 > 7	True
4 < 9	True
8 >= 8	True
3 <= 1	False
'Python' == 'Python'	True
'Python' != 'java'	True

6 Key Takeaways

- [x] Comparison operators return only True or False.
- [x] They can be used with numbers, strings, and variables.
- [x] They are commonly used in if conditions and loops.
- [x] Use == to check equality, not = (which is assignment).
- [x] Combine with logical operators (and, or, not) for complex conditions.

Did You Know?

Comparison operators can also be used with strings based on alphabetical order!

Example: 'apple' < 'mango' → True

(Because 'a' comes before 'm')

Code Practice Repeat

"Compare, decide, create better programs."

– Satyarthi

Keep Learning Keep Growing

Python String Methods

Practice Learn Improve

Practice Questions – Test Your Understanding!

Question 1

Topic: **strip()** method

What will be the output of the following code?

```
s = "   Python Programming   "
print(s.strip())
```

A ' Python Programming '

B 'Python Programming'

C 'python programming'

D 'Python'

Answer: B

Explanation:

The **strip()** method removes leading and trailing spaces from the string. So, the output is **'Python Programming'** (without extra spaces).

Question 2

Topic: **title()** method

What will be the output of the following code?

```
s = "hello world from python"
print(s.title())
```

A 'Hello world from python'

B 'Hello World From Python'

C 'HELLO WORLD FROM PYTHON"

D 'hello world from python'

Answer: B

Explanation:

The **title()** method converts the first letter of each word to uppercase and the remaining letters to lowercase.
So, the output is **'Hello World From Python'**.

"Small Steps Big Skills!"

Practice Strings, Build a Stronger Python!

By Satyarthi